Finding the Bayes Net in the Transformer: A Tutorial on Message Passing, Belief Propagation, and Transformer Circuits

Greg Coppola
coppola.ai greg@coppola.ai

2026

Abstract

A sigmoid transformer can exactly realize belief propagation computations. This is not a loose analogy or a high-level functional similarity: under the constructive mapping proved in the companion paper [3], the transformer forward pass implements the BP update equations directly. This paper is a tutorial guide to what that result means, how it connects to transformer components, and what it implies for understanding trained models. We walk through the transformer architecture component by component: residual streams as evolving belief states, attention as evidence routing, feed-forward networks as belief updates, and layers as rounds of message passing. We develop the underlying log-odds algebra of Turing, Good, and Pearl — the mathematical spine connecting all three — and show that five independent research programs in graphical models, mechanistic interpretability, in-context learning, graph neural networks, and constructive weight proofs have converged on the same picture from different starting points. The companion paper [3] develops the formal constructive theorems. This paper is the conceptual and mechanistic guide to the broader picture those theorems open up.

Contents

Part I
The Mathematical Foundation

1 Introduction

A sigmoid transformer can exactly realize belief propagation computations. This is not a loose analogy or a high-level functional similarity: under the constructive mapping proved in the companion paper [3], the transformer forward pass implements the BP update equations directly. The present paper is a tutorial guide to what that result means, how it connects to transformer components, and what it implies for understanding trained models.

The formal anchor is the companion theorem. This paper does not rederive it. Instead, it builds the conceptual picture around it: opening each transformer component, identifying its role in the message-passing account, and connecting the formal result to a converging body of empirical work in mechanistic interpretability, in-context learning, and graphical model theory.

Scope of the Claims

The paper operates at four distinct levels:

1. **Exact constructive results.** The companion theorem gives an exact realization of BP in sigmoid transformers under a constructive weight mapping [3].

2. **Empirical observations.** Mechanistic interpretability results from trained transformers exhibit structures consistent with the BP interpretation.

3. **Mechanistic interpretation.** Transformer components can be understood through a routing-and-inference decomposition aligned with BP primitives.

4. **Speculative extensions.** The discussions of grounding, hallucination, and hybrid continuous-discrete inference are interpretive extensions rather than established theorems.

These claims have different evidentiary status and are marked accordingly throughout the paper.

The Five Correspondences

The tutorial is organized around five structural correspondences between transformer components and the operations of belief propagation:

1. **The residual stream is the belief state.** Each token's D_{model}-dimensional vector stores the current accumulated state of inference at that node. Residual connections preserve and update beliefs across layers rather than replacing them outright.

2. **Attention performs evidence gathering.** Attention heads route information from neighboring tokens into the residual stream before the update step executes. Multiple required evidence sources become jointly available in a shared workspace.

3. **The FFN performs belief update.** Under the constructive BP mapping, the feed-forward network computes updateBelief$(m_0, m_1) = \sigma(\text{logit}(m_0) + \text{logit}(m_1))$ from the gathered evidence. This is the Bayesian update rule for independent binary evidence sources — exact with sigmoid activation, approximate with ReLU or SwiGLU.

4. **Layers correspond to rounds of message passing.** One layer is one gather-then-update cycle. Stacking layers corresponds naturally to iterative rounds of inference.

5. **The scaling dimensions describe inference capacity.** The architectural dimensions — W, L, H, D_{ff}, D_{model} — determine the scale of the routing and inference operations available to the network.

The Running Example

Throughout the paper we will repeatedly use a simple example involving three propositions:

`like(jack,jill)`, `like(jill,jack)`, `date(jack,jill)`.

The example is intentionally small. Its purpose is not realism but clarity. We use it repeatedly to illustrate how attention gathers evidence, how FFNs combine beliefs, how residual streams store intermediate state, how layers propagate information over multiple rounds, and what grounding means in a formally declared concept space. By keeping the example fixed across sections, the paper can focus on computational structure rather than constantly introducing new notation.

The Underlying Algebra

All five correspondences rest on the same mathematical spine: the log-odds algebra of independent binary evidence, developed by Turing and Good [8], formalized by Pearl as the sum-product algorithm [13], and realized exactly by the constructive sigmoid mapping. Within this mapping, sigmoid is the exact activation required to recover the log-odds update equations. Part I develops this foundation before turning to the architectural correspondences.

Grounded and Ungrounded Inference

The distinction between grounded and ungrounded transformers is not primarily architectural. In both cases, the network performs inference over an implicit factor graph. The difference is whether the graph has a declared ontology, finite concept space, and verifier that permits formal correctness guarantees.

For grounded transformers operating on tree-structured knowledge bases, the constructive BP results provide exact posterior guarantees. Standard language models do not provide guarantees of this kind because their internal representations are not tied to a formally declared ontology. Part IV develops this distinction and its implications for hallucination and reliability.

Convergent Evidence

Five independent research programs have converged on the same general picture from different starting points: the transformer forward pass implements a form of distributed probabilistic inference. Mechanistic interpretability studies identify routing heads and structured FFN updates. Belief-state geometry work finds posterior-like organization in residual streams. In-context learning work models transformers as Bayesian learners. Graph neural network research established the message-passing connection. Constructive weight-learning experiments show that optimization reliably discovers BP-like solutions. Part V surveys this convergence.

How to Read This Paper

Part I establishes the mathematical foundation. Readers already familiar with log-odds algebra and belief propagation can skim portions of it. Part II develops the core architectural correspondences component by component. Part III examines superposition, the three softmaxes, and the relationship between trained weights and implicit factor graphs. Part IV addresses grounding and hallucination. Part V surveys convergent evidence and related work.

The formal constructive results are developed in [3]. This paper is a tutorial guide to the conceptual landscape those results open up.

2 The Log-Odds Algebra

The mathematical spine of the entire account is a single algebraic structure: the group of beliefs on $(0, 1)$ under log-odds addition. This algebra was discovered three times independently, in three different contexts, by three different research traditions. Each time, the same formula emerged.

First Discovery: Turing and Good at Bletchley Park

During World War II, Alan Turing and I.J. Good needed a practical method for combining independent pieces of evidence when breaking Enigma. The method they developed was log-odds addition [8].

Given a binary hypothesis H and independent evidence sources e_0, e_1, the weight of evidence adds:

$$\text{logit}(P(H \mid e_0, e_1)) = \text{logit}(P(H)) + W(H : e_0) + W(H : e_1).$$

Starting from a uniform prior $P(H) = 0.5$, so $\text{logit}(P(H)) = 0$, and setting $m_i = P(H \mid e_i)$:

$$\text{logit}(P(H \mid e_0, e_1)) = \text{logit}(m_0) + \text{logit}(m_1).$$

Converting back via sigmoid: $P(H \mid e_0, e_1) = \sigma(\text{logit}(m_0)+\text{logit}(m_1)) = \text{updateBelief}(m_0, m_1)$.

The updateBelief function is Turing and Good's weight-of-evidence combination, applied to two binary evidence sources. It is not an approximation. For independent evidence sources, it is exact.

Second Discovery: Pearl's Sum-Product Algorithm

Pearl [13] derived the belief propagation update rule for binary variable nodes from the general sum-product equations applied to pairwise factors with independent messages. The result is:

$$b_{\text{new}}(v) = \frac{m_0 m_1}{m_0 m_1 + (1 - m_0)(1 - m_1)} = \sigma(\text{logit}(m_0) + \text{logit}(m_1)).$$

Pearl derived this from the structure of graphical models. He did not frame it as log-odds addition or connect it to Turing and Good's weight-of-evidence tradition. The formula is the same. The framing is new.

Pearl's deeper contribution was showing that this update rule, applied locally at each node using only messages from immediate neighbors, is sufficient to compute exact marginal posteriors on tree-structured graphs [13]. Global inference emerges from local computation. No central coordinator is needed: each node updates its belief from its neighbors' current beliefs, and iterating this process to convergence yields the exact answer. This is the sum-product algorithm, and it is what makes the connection to transformers non-trivial. A transformer is also a distributed computation in which each position updates from its neighbors across layers, with no global state outside the residual stream. The architectural parallel is not incidental — it is the reason the same formula appears in both systems.

Third Discovery: The Sigmoid Transformer

The sigmoid FFN computing $\sigma(w_0 \cdot \text{logit}(m_0) + w_1 \cdot \text{logit}(m_1) + b)$ is performing weighted log-odds addition and converting back to probability space in a single operation. With $w_0 = w_1 = 1$ and $b = 0$, this is updateBelief exactly. The sigmoid activation is not a design choice motivated by gradient flow or output normalization. It is the exact function required to implement the Turing-Good-Pearl algebra.

The Algebraic Structure

Log-odds addition defines a commutative group on $(0, 1)$ via the isomorphism $\text{logit} : (0, 1) \to \mathbb{R}$:

$$m_0 \oplus m_1 = \sigma(\text{logit}(m_0) + \text{logit}(m_1)).$$

- **Commutativity:** $m_0 \oplus m_1 = m_1 \oplus m_0$.
- **Associativity:** $(m_0 \oplus m_1) \oplus m_2 = m_0 \oplus (m_1 \oplus m_2)$.
- **Identity:** $m \oplus 0.5 = m$, since $\text{logit}(0.5) = 0$.
- **Inverse:** $m \oplus (1 - m) = 0.5$, since $\text{logit}(m) + \text{logit}(1 - m) = 0$.

The identity element is 0.5 — maximum uncertainty, no information. Combining any belief with a 0.5 belief leaves it unchanged. This is the formal statement that a neutral prior contributes nothing.

Relation to Classical Boolean Logic

This algebra is the probabilistic generalization of classical Boolean AND and OR. In the limit as beliefs approach 0 and 1:

- $\text{logit}(1) = +\infty$ (certain true), $\text{logit}(0) = -\infty$ (certain false).
- Two large positive numbers sum to a larger positive number: high confidence in both inputs yields high confidence in their conjunction.
- Equal and opposite evidence cancels: $\text{logit}(p) + \text{logit}(1-p) = 0$, yielding 0.5 — maximum uncertainty.

Classical Boolean logic is the limiting case as beliefs become certain. The log-odds algebra does not have an annihilator (FALSE AND TRUE $\neq$ FALSE in the probabilistic case); instead it has cancellation: strong evidence in opposite directions yields maximum uncertainty. This is the correct behavior for uncertain reasoning.

Why This Matters

The three independent rediscoveries of the same formula are not a coincidence. The log-odds algebra is the unique correct way to combine independent binary evidence under the axioms of probability theory. Any system that does probabilistic reasoning over binary propositions with independent evidence sources will converge on this algebra. The transformer converged on it because it was trained to do probabilistic reasoning. Gradient descent did not discover something new. It rediscovered the structure that the analysis of reasoning already required.

3 Two Theorems, One Weight Construction

Before developing the five correspondences in detail, it is worth pausing on a structural fact that makes the BP interpretation more than a clever observation. The transformer's routing mechanism is so clean that the *same two weight families* appear in two completely independent proofs: the Turing completeness proof and the BP proof. This is not a coincidence. It reveals that the projectDim/crossProject construction is the natural primitive for what attention heads actually do.

Turing Completeness

Pérez et al. [14] proved that transformers are Turing complete under floating-point arithmetic. The companion paper [3] gives a constructive Lean-verified proof via Boolean circuit simulation:

Theorem 3.1 (Transformer is Turing Complete)**.** *For any Turing machine M, there exist transformer weights W such that for all tapes τ and step counts t:*

$$\text{transformerAgent}(W, \text{encode}(M, \tau), t) = \text{encode}(M.\text{run}(\tau, t)).$$

Formally verified in Lean 4 against standard mathematical axioms.

The proof is constructive. Any Turing machine step decomposes into a Boolean circuit. A transformer simulates any Boolean circuit in a single FFN forward pass via threshold gates. The key insight is the role of each component: *attention is lookup* (find the tape cell at the current head position), *FFN is gating* (apply the transition function), and *the agent loop is iteration* (step the machine).

The Same Two Weight Families

The Turing completeness proof and the BP proof use the same two sparse matrix families for Q/K and V weights:

- projectDim(d): a rank-1 diagonal matrix extracting dimension d. Used as W_Q and W_K. The attention score $e_j[d] \cdot e_k[d]$ peaks when token k's stored index matches token j's query value — exact index matching. In the TM proof: finds the tape cell at the head position. In the BP proof: finds the neighbor node whose belief to gather.
- crossProject(s, d): a rank-1 off-diagonal matrix reading source dimension s and writing to destination dimension d. Used as W_V. In the TM proof: routes the tape symbol to the transition function. In the BP proof: routes the neighbor's belief into the residual stream scratch slot.

Both proofs also reuse two results from the Turing completeness Lean development: `posEncDot_distinct` (the $Q \cdot K$ score is strictly maximized at the correct target) and `softmax_concentrates` (at sufficient temperature, softmax converges to a point mass at the argmax).

What This Means

The shared weight construction is significant for two reasons.

First, it means the routing mechanism is *canonical*. There is one natural way to make attention heads perform index-based lookup in a transformer, and both the Turing and BP interpretations use it. This is not a special-case construction tailored to BP — it is the architecture's natural primitive.

Second, it separates routing from inference. The projectDim/crossProject construction handles routing (which token to look at) identically in both proofs. What differs is what happens *after* the lookup: the TM proof uses the FFN as a Boolean gate; the BP proof uses it as a probabilistic update. Same routing, different inference. The transformer is a general-purpose routing-plus-inference architecture, and both Turing completeness and BP are natural instances of it.

The Turing completeness result says: the transformer *can* compute anything. The BP result says: it *does* compute something specific — belief propagation — with these specific weights. Both are needed for a complete picture of what the architecture is capable of and what it actually does.

Empirical Confirmation

The `learner` repository confirms that gradient descent finds TM-simulation weights from scratch. Five structurally distinct Turing machines — binary incrementer, decrementer,

bitwise complement, left shift, right shift — were trained using a standard transformer encoder (2 layers, 2 heads, $d_{\text{model}} = 32$, ∼4K parameters) from random initialization. All five reached 100% validation accuracy in 4 epochs with identical hyperparameters and no per-machine tuning [3].

The formal construction is not merely theoretically possible — it is what gradient descent finds given a clean training signal. This parallels the `bayes-learner` result for BP (val MAE 0.000752), which we develop in Section 6. Together they establish a consistent pattern: the formal weight constructions are what the architecture naturally learns.

Part II
The Five Correspondences

4 The Residual Stream as the Belief State

The starting point for seeing the Bayes net in the transformer is the residual stream. Everything else follows from getting this right.

In the standard transformer encoder, each token position carries a vector of dimension D_{model} that persists across all layers. Each layer reads from this vector and writes its output back into it via a residual connection. Nothing is discarded; the stream accumulates. Elhage et al. [5] name this the *residual stream* and frame it as a shared workspace: attention heads and feed-forward layers are read-write operations on a common scratchpad.

In the Bayes net view, the residual stream at token position t is the *belief state* of factor graph node t. Specifically, the formal token encoding from [3] assigns clean semantic roles to each dimension:

Dimensions	Content
0	own belief (initialized to 0.5)
1–4	factor table entries $[f_{00}, f_{01}, f_{10}, f_{11}]$
5	node type (0 = variable, 1 = factor)
6	own index $/(n-1)$
7	neighbor index $/(n-1)$

Dimensions 5–7 form the *routing key*: the complete inferential identity of the token. Dimensions 0–4 are continuous parameters: they supply the magnitudes the inference computes over, but they do not determine the structure of the computation. This split is not a convenience of presentation — it is a theorem. Two tokens with the same routing key receive identical attention patterns regardless of their continuous parameters [3].

Routing Key = Concept. Continuous Dimensions = Magnitude.

This is the right level of abstraction for thinking about what the residual stream represents. The routing key identifies *which* concept a token corresponds to. The continuous dimensions encode *how strongly* the evidence supports that concept. A production transformer with $D_{\text{model}} = 4096$ is not tracking 4096 independent propositions — it is tracking a much smaller

number of concepts (bounded by the routing key space) while using the remaining dimensions to encode magnitudes and handle superposition.

The residual connection is what makes this interpretation coherent across layers. Belief propagation is an iterative algorithm: each round refines the belief at every node by incorporating messages from neighbors. The residual connection ensures that each layer's update is added to the accumulated belief, not computed from scratch. Layer l does not replace the belief; it refines it.

What This Looks Like in Trained Models

The mechanistic interpretability literature has independently converged on a compatible picture. Elhage et al. [4] show that the residual stream represents far more features than D_{model} dimensions via superposition: features are encoded as nearly-orthogonal directions in a high-dimensional space, with interference managed by sparsity. In the Bayes net view, this means the residual stream is simultaneously tracking many factor graph nodes, each encoded as a direction rather than a dedicated dimension.

Sparse autoencoder (SAE) features [18] are the natural candidates for individual factor graph nodes. A monosemantic SAE feature — one that activates for a single coherent concept — corresponds to a single routing key in the formal construction. The feature direction in residual stream space is where the belief value for that concept lives.

The broader implication: the residual stream is not a flat representation. It has structure — routing structure, imposed by the attention weights, that determines which features interact with which. That structure is the implicit factor graph.

5 Attention as Evidence Gathering

Within the BP interpretation, the attention mechanism performs the gather phase of message passing: it routes neighboring beliefs into the residual stream before the update step executes. This corresponds to the evidence-gathering phase of Pearl's sum-product algorithm [13].

The conjunctive structure is architectural rather than localized inside any individual attention head. Consider a single transformer layer. Each attention head scans token positions, concentrates on a relevant neighbor, and copies information from that neighbor into a designated region of the current token's residual stream. Only after all heads have finished does the feed-forward network execute.

The conjunction is therefore not inside any one head. It emerges at the level of the residual stream, where multiple required evidence sources become simultaneously available in a shared workspace before the update step runs. The FFN has no mechanism to execute on partial information: it reads the entire residual stream state. In this sense, the architecture enforces a gather-before-update semantics analogous to conjunctive evidence integration.

The Formal Weight Construction

The constructive mapping from [3] realizes the gather step exactly. Each attention head uses two sparse weight families:

- projectDim(d): a diagonal projection extracting dimension d. Used as W_Q and W_K. The resulting $Q \cdot K$ dot product peaks when token k's stored index matches token j's query value, implementing exact index matching.
- crossProject(s, d): an off-diagonal projection reading source dimension s and writing to destination dimension d. Used as W_V. Routes neighbor k's belief from one residual-stream dimension into another.

At sufficiently high temperature, the softmax concentrates almost entirely on the matching neighbor. In the constructive setting, sharp attention behaves like a routing or lookup operation rather than an inference computation. The probabilistic update occurs in the FFN stage.

A Concrete Example

Consider three propositions: `like(jack,jill)` with belief $m_0 = 0.8$, `like(jill,jack)` with belief $m_1 = 0.4$, and `date(jack,jill)` whose belief is to be updated.

Head 0 attends to `like(jack,jill)` and writes 0.8 into one region of the `date` token's residual stream. Head 1 attends to `like(jill,jack)` and writes 0.4 into another region. By the time the FFN executes, both evidence sources are simultaneously available. The gather phase is complete before the update step begins.

Evidence from Trained Models

Mechanistic interpretability studies have identified attention patterns consistent with the routing behavior predicted by the constructive mapping. Wang et al. [20] identify "name mover heads" in GPT-2 small that copy token-specific information to target positions, behavior closely resembling crossProject-style routing. Their Q/K circuits also exhibit approximately rank-1 structure, qualitatively consistent with projectDim-style matching.

Empirical classification studies further identify several recurring head types:

- **Semantic routing heads**: attend to semantically related tokens determined by content rather than position. These resemble the gather step of the constructive BP mapping.
- **Hub heads**: attend consistently to fixed positions, especially the BOS token. These appear to distribute or collect global context information through a shared communication channel.

These observations do not by themselves prove the BP interpretation, but they are structurally consistent with a routing-plus-update view of transformer computation.

Scaling

For Llama 3 405B, there are $L \times H = 126 \times 128 = 16{,}128$ distinct attention-head routing patterns. The number of routing operations per forward pass scales as $L \times H \times W = 132\text{M}$ at sequence length $W = 8192$. Within the BP interpretation, these routing operations form the communication structure of the implicit factor graph.

6 The FFN as Belief Update

Once attention has assembled the relevant evidence into the residual stream, the feed-forward network performs the update step. In the exact constructive setting, this update is the binary belief-propagation rule for combining independent evidence sources.

The central formula is:

$$\text{updateBelief}(m_0, m_1) = \frac{m_0 m_1}{m_0 m_1 + (1 - m_0)(1 - m_1)} = \sigma(\text{logit}(m_0) + \text{logit}(m_1)) \,.$$

With sigmoid activation and BP weights ($w_0 = w_1 = 1$, $b = 0$), the FFN computes this exactly from the gathered beliefs in dimensions 4 and 5, writing the result to dimension 0. This is the Bayesian update rule for two independent binary evidence sources in the constructive model [3].

The Boolean Structure Inside the Update

The formula contains the same structure that appears in classical Boolean reasoning, but in probabilistic form. The term $m_0 m_1$ is the joint probability that both independent inputs are true. In log-odds space, this multiplication becomes addition:

$$\text{logit}(\text{updateBelief}(m_0, m_1)) = \text{logit}(m_0) + \text{logit}(m_1).$$

Independent evidence adds. The sigmoid converts the log-odds sum back to probability space. The FFN is therefore a weight-of-evidence combiner in the sense of Turing and Good [8], when its inputs and weights instantiate the constructive BP representation.

This is why the OR-gate language is useful: the FFN combines gathered evidence into an updated conclusion. But the exact theorem is more specific than the metaphor. The FFN implements the BP update exactly in the sigmoid construction; production FFNs approximate or generalize this role rather than literally instantiating the same binary rule in every unit.

The Key-Value Memory Interpretation

Geva et al. [6] show that FFN layers can be viewed as key-value memories: the rows of W_1 are keys that detect input patterns, and the columns of W_2 are values written to the residual stream when those patterns activate. Each hidden unit detects a pattern and contributes an update.

In the BP interpretation, these key-value memories resemble learned factor potentials. The key determines when an update is relevant; the value determines what direction is written back into the residual stream. The full FFN layer integrates many such updates into the token's next residual-stream state.

ROME [11] provides suggestive evidence for this interpretation: editing localized FFN parameters can change factual associations in targeted ways. This does not by itself prove that FFN weights are literal conditional-probability tables, but it is consistent with the view that FFNs store semantically structured update directions.

Sigmoid vs. ReLU

The formal proof requires sigmoid activation because updateBelief is exactly $\sigma(\text{logit}(m_0) + \text{logit}(m_1))$: sigmoid is the inverse of logit, and the composition gives the desired update. ReLU, GeLU, and SwiGLU do not implement this binary log-odds algebra exactly.

Nevertheless, these activations may still support approximate or generalized message-passing behavior. They preserve smooth, structured transformations of residual-stream features, and empirical experiments show that gradient descent can find approximately correct BP-like solutions even with ReLU activations. In the `bayes-learner` experiment, for example, the learned model reached validation MAE 0.000752 [3].

Thus sigmoid gives the exact algebraic identity. Modern activations give a practical approximation or extension, not the same formal guarantee.

Scaling

For Llama 3 405B, there are $L \times D_{ff} = 126 \times 53{,}248 \approx 6.7\text{M}$ FFN hidden units across layers. The number of FFN evaluations per forward pass is $L \times D_{ff} \times W \approx 55\text{B}$ at sequence length $W = 8192$.

Within the BP interpretation, this large FFN budget reflects the cost of implementing rich update rules over many superposed features. The architecture devotes far more capacity to updating internal state than to routing attention alone, which is consistent with the view that inference is the dominant computational burden.

7 Layers Are Rounds of Belief Propagation

One transformer layer is one round of belief propagation. L layers are L rounds. This is the content of Theorem 1.1 in [3], and it is the claim that ties the previous three sections together.

A single round of BP proceeds in two steps. In the gather step, each variable node copies its neighbors' current beliefs into scratch slots. In the update step, each variable node computes a new belief from the gathered evidence. One transformer layer does exactly this: attention is the gather step, the FFN is the update step, and the strict alternation is architecturally enforced. The depth of the network is not an engineering hyperparameter — it is determined by the number of reasoning hops required.

The Peirce Lower Bound

No local algorithm can solve N-hop reasoning in fewer than N rounds. This is the Peirce lower bound, and it applies directly to belief propagation: on a tree of diameter d, BP requires exactly d rounds to propagate evidence from one end to the other [13]. A transformer with L layers can perform at most L hops of reasoning. More layers are required for longer reasoning chains, not for more parameters per se.

This reframes the question of why large models need depth. It is not that deeper models have more capacity in a vague sense. It is that deeper models can perform more rounds of inference — they can follow longer chains of implication. A model that needs to conclude $A \Rightarrow B \Rightarrow C \Rightarrow D$ needs at least three layers, regardless of how wide each layer is.

Three-Phase Layer Structure

Experiments classifying attention heads in GPT-2 small by behavior reveal a three-phase structure across layers:

- **Layers 0–2 (continuous-heavy):** local feature extraction. Attention heads attend to nearby tokens; the residual stream accumulates basic syntactic and semantic features. This is the first round of BP: each node gathers information from its immediate neighbors.
- **Layer 3 (transition):** a shift in the dominant computation. Hub heads begin to dominate. The BOS token starts accumulating a global context representation.
- **Layers 4–11 (hub-dominated):** global context assembly. Hub heads distribute the BOS representation to every token. This is iterative refinement of a global prior — each round incorporates the accumulated evidence from all previous rounds.

The semantic-AND heads do not scale in number: 9 in GPT-2 small, 9 in GPT-2 medium. What scales is communication infrastructure (hub heads), not logical reasoning capacity per layer. This is consistent with the BP view: the number of reasoning steps is set by depth, not width.

Residual Connections as Belief Accumulation

The residual connection has a natural interpretation in this account. Each layer adds its update to the accumulated belief, rather than replacing it. This is exactly the structure of iterative BP: each round refines the beliefs from the previous round. The residual stream after l layers encodes the belief state after l rounds of message passing.

Without residual connections, each layer would compute its output from scratch and the iterative refinement interpretation would break down. With residual connections, the stream has a well-defined semantic content at every depth: it is the belief state of the implicit factor graph after that many rounds of BP.

Convergence

On tree-structured factor graphs, BP converges in exactly diameter(T) rounds and the resulting beliefs are the exact marginal posteriors. This is the no-hallucination corollary of [3]: a transformer with BP weights, run for sufficiently many layers over a grounded tree-structured knowledge base, computes exact Bayesian posteriors at every node.

On loopy graphs, BP may not converge, but in practice it does on the graph structures that arise in typical knowledge bases. The `loopy` repository reports 100% convergence across 500 trials on five loopy graph structures, with mean KL divergence below 0.0002 [3]. The theoretical gap between trees and loopy graphs is smaller in practice than in the worst case.

8 The Scaling Numbers as a Bayes Net Inventory

The five architectural dimensions of a transformer — W (sequence length), D_{model} (embedding dimension), H (attention heads), D_{ff} (FFN hidden dimension), L (layers) — are not arbitrary

engineering choices. In the Bayes net view, each dimension has a precise interpretation, and together they constitute an inventory of the implicit factor graph.

Dimension	BP interpretation	Llama 3 405B
W	nodes in the factor graph (sequence positions)	8,192
L	rounds of belief propagation	126
H	AND patterns per layer	128
D_{ff}	OR patterns per layer	53,248
$L \times H$	distinct AND weight patterns	16,128
$L \times D_{ff}$	distinct OR weight patterns	6.7M
$L \times H \times W$	AND evaluations per forward pass	132M
$L \times D_{ff} \times W$	OR evaluations per forward pass	55B

The ratio $D_{ff}/H \approx 416$ for Llama 3 405B. The architecture devotes 416× more capacity to inference (OR gates) than to routing (AND gates). This is the right ratio for a system whose bottleneck is combining evidence, not finding it.

What Scaling Adds

When a model scales from 8B to 405B parameters, what changes in Bayes net terms?

- **More rounds of BP** (L increases): deeper reasoning chains become possible. A 126-layer model can follow implications 126 hops deep.
- **Richer OR patterns** (D_{ff} increases): more conditional probability table entries per layer. The model can represent more fine-grained conditional relationships.
- **More nodes** (W increases): longer sequences mean larger factor graphs. The model reasons over more propositions simultaneously.
- **AND patterns stay small** (H grows slowly): 128 heads in 405B vs. 16 in the smallest models. The number of distinct routing operations is nearly constant. What scales is inference capacity, not routing capacity.

The implication is sharp: scaling adds inference depth and richness, not fundamentally new reasoning structure. The AND/OR architecture is fixed. What changes is how many rounds it runs and how many patterns it can match.

The 16,128 Number

The 16,128 distinct AND weight patterns for Llama 3 405B is a striking number because it is so small. Every token position in every sequence uses one of these 16,128 routing patterns — the same patterns, regardless of what the token is. The routing key (which neighbor to attend to) is determined by the weight matrices, which are shared across positions. What varies across positions is the continuous content of the residual stream, not the routing structure.

This is the formal statement of weight sharing as universal rule implementation (Section 4.3 of [3]): the same computation runs for every instantiation of a concept, and the particular

input supplies the magnitudes. The 16,128 patterns are the universal rules. The residual stream values are the particular instances.

Part III
Going Deeper

9 The Residual Stream vs. Propositions: Superposition and the D_{ff}/D Ratio

The formal BP construction uses $D_{\text{model}} = 8$ with one dedicated dimension per proposition and one OR gate per belief update. A production transformer has $D_{\text{model}} = 4096$ and $D_{ff} = 14{,}336$: approximately 3.5 OR gates per residual stream dimension per layer. This raises two related questions. Why does the ratio D_{ff}/D exceed 1 at all? And does $D_{ff} \approx 14{,}336$ mean there are 14,336 factor nodes per layer? This section resolves both.

Hidden Units Are Not Factor Nodes

In the minimal construction, one FFN hidden unit implements one factor node — one conditional relationship between two propositions. Hidden units and factor nodes are in one-to-one correspondence.

In a production transformer, this correspondence breaks down in two ways:

1. **Multiple units per factor.** A single conditional relationship may require multiple hidden units to represent accurately with non-sigmoid activations. ReLU and SwiGLU require more units to approximate the exact log-odds combination that sigmoid implements in one unit. The factor node is still the unit of semantics; the hidden units are the implementation substrate.

2. **Superposed factors.** Because the residual stream represents many concepts via superposition, a single hidden unit may participate in multiple factor relationships simultaneously. The unit fires for a direction in residual stream space that is a superposition of several feature directions, and the contribution it writes is a superposition of several belief updates.

The right level of abstraction is not hidden units but *SAE features*. Each monosemantic SAE feature corresponds to one factor node in the implicit factor graph. The number of semantically distinct factor nodes per layer is the number of active SAE features — typically much smaller than D_{ff}.

D_{ff} is therefore the *implementation budget* for the OR gates, not the factor node count. It sets an upper bound on representable conditional relationships per layer, but the actual semantic content is determined by the SAE feature structure.

Three Reasons the Budget Exceeds One

Three independent factors drive D_{ff}/D above the minimal value of 1:

Superposition. Elhage et al. [4] show that residual streams represent far more features than D_{model} dimensions via superposition: features are encoded as nearly-orthogonal directions in a high-dimensional space, with interference managed by sparsity. A model with $D = 4096$ may track tens of thousands of distinct concepts simultaneously. If the residual stream tracks K active concepts per token and each needs an independent belief update per layer, D_{ff} must be at least K. Since $K \gg D$, the OR budget must exceed the residual stream dimension by the superposition density — empirically, roughly 3–4×.

k-Ary neighbors. The formal construction handles pairwise factors: each OR gate takes exactly two inputs. Real knowledge graph nodes have more than two neighbors. The scaling theorem of [3] handles this: any k-ary factor graph binarizes exactly via a chain of $\lceil \log_2 k \rceil$ pairwise updates. A node with 8 neighbors requires 3 chained pairwise updates rather than 1, contributing a further multiplicative overhead to the required D_{ff}.

Activation precision. Sigmoid implements the log-odds algebra exactly. ReLU and SwiGLU are compatible but not exact: they preserve non-negativity and smooth aggregation but require more hidden units to achieve the same approximation quality. The bayes-learner experiment confirms that gradient descent finds approximately correct BP solutions even with non-sigmoid activations (val MAE 0.000752), but at the cost of additional hidden units per factor.

Together these three factors explain why the minimal $D_{ff}/D = 1$ becomes 3–4 in production models. The ratio is an architectural constant encoding the combined implementation overhead of superposition density, neighbor fanout, and activation imprecision.

The Dimension Budget and Factor Graph Scale

A cleaner way to think about the implicit factor graph uses the dimension budget directly. The residual stream has D_{model} dimensions. Each active concept occupies a direction in this space. Empirically, roughly $10 \times D_{\text{model}}$ features can be superposed with manageable interference [4].

For Llama 3 405B with $D_{\text{model}} = 16{,}384$, this gives roughly 160,000 simultaneously active features per token position. Each feature is a factor graph node. The factor graph has on the order of $W \times 160{,}000 \approx 1.3$B nodes for a sequence of length 8192. This is the scale of the implicit Bayesian network that one forward pass of Llama 3 405B is performing inference over.

The OR/AND Asymmetry

With this clarification, the $D_{ff}/H \approx 416$ ratio for Llama 3 405B has a clean interpretation. It is not that there are 416 OR gates for every AND gate. It is that the OR implementation requires 416 units of hidden dimension for every unit of attention head dimension. The factor graph itself may be much sparser — each node may have only 2–5 neighbors in practice — but implementing those sparse connections accurately in the superposed representation requires the 416× overhead.

The implication for interpretability: finding the factor graph requires working at the SAE feature level, not the hidden unit level. The factor potentials are encoded in the relationship between input SAE features (what the W_1 rows respond to) and output SAE features (what the W_2 columns write). The D_{ff} hidden units are the implementation substrate; the SAE features are the semantics.

10 The Three Softmaxes

There are three places in the transformer where softmax or sigmoid appears. They do three completely different jobs. Conflating them is the single most common source of confusion about what the transformer is computing.

	Job	Input	Semantics
Softmax 1	Routing	$Q \cdot K$ scores	Differentiable argmax
Softmax 2	Inference	Neighbor beliefs	Bayesian posterior
Softmax 3	Generation	Final hidden state	Token distribution

Softmax 1: Attention (Routing)

The first softmax appears inside every attention head, applied to the query-key dot products. Its job is routing: which token should I look at? This is a differentiable argmax — at high temperature it concentrates all weight on the token with the highest $Q \cdot K$ score.

In the BP construction, the attention softmax fetches a neighbor's belief by index matching. The query encodes "I am looking for the token at index i"; the key encodes "I am the token at index i"; the dot product peaks at the correct match; softmax concentrates the weight there. The result is a lookup table.

This softmax is *not* doing inference. It is not computing a probability over hypotheses. It is selecting which piece of evidence to retrieve. Calling it a "probability distribution over tokens" is technically correct but misleading — the weights are routing coefficients, not posterior beliefs.

Softmax 2: The FFN Sigmoid (Inference)

The second softmax is the sigmoid activation in the FFN. Its job is Bayesian inference: what is my updated belief given my neighbors' beliefs?

$$\text{updateBelief}(m_0, m_1) = \sigma(\text{logit}(m_0) + \text{logit}(m_1)) \,.$$

This is where the actual Bayesian computation happens. The sigmoid is the exact inverse of logit, converting the log-odds sum back to a probability. This is the weight-of-evidence combination of Turing and Good [8], applied to two binary evidence sources.

This softmax *is* computing a posterior. The output is a belief in $(0, 1)$ representing the updated probability that the current proposition is true, given the gathered evidence. This is the inference step. Everything else in the transformer is infrastructure for getting the right inputs to this computation.

Softmax 3: Output (Generation)

The third softmax appears at the output layer, applied to the logits over the vocabulary. Its job is generation: which token comes next? In a standard language model, the logits are computed by a matrix multiply over the final hidden state — pattern-matched association rather than computed inference.

In a grounded QBBN transformer, the output would be different: the logits would come from computed BP posteriors, representing the marginal probability of each proposition being true given the evidence. The softmax operation is the same; what changes is what it is applied to.

The QBBN does not replace the softmax. It replaces the logits. The architecture was always capable of computing exact posteriors; it just needed the right weights to feed into the output softmax. Standard language model training produces weights that implement pattern-matched association. BP training would produce weights that implement principled inference.

Why the Distinction Matters

Conflating Softmax 1 and Softmax 2 leads to the mistaken conclusion that the attention weights are the inference. They are not. The attention weights determine *which* evidence to gather. The FFN sigmoid determines *what to conclude* from that evidence. Routing and inference are separate computations, performed by separate components, in a strict temporal order enforced by the architecture.

Conflating Softmax 2 and Softmax 3 leads to the mistaken conclusion that the output distribution is a posterior over propositions. In an ungrounded model, it is not — it is a distribution over next tokens learned by association. The two converge only in a grounded model where every output token corresponds to a declared proposition with a known prior.

11 How to Read the Weights

The previous sections have established what the transformer components *are* in Bayes net terms. This section asks a more practical question: given a trained transformer, how would you actually find the factor graph? What would you look for in the weight matrices?

The formal construction gives precise predictions. A transformer implementing exact BP has weight matrices with specific sparse structure. A trained transformer implementing approximate BP should show approximately this structure. This section makes the predictions concrete enough to test.

What to Look for in Attention Weights

The formal AND construction uses two weight families for each head:

$$W_Q = W_K = \text{projectDim}(d), \qquad W_V = \text{crossProject}(s, d).$$

projectDim(d) is a rank-1 matrix with a single nonzero entry on the diagonal at position (d, d). crossProject(s, d) is a rank-1 matrix with a single nonzero entry at position (d, s).

The predictions for a semantic-AND head in a trained model:

1. **The Q/K circuit is approximately rank-1.** Compute the effective Q/K matrix $W_Q^T W_K$ (or its low-rank approximation). A BP head should have most of its singular value mass in a single component. The dominant left and right singular vectors should correspond to the same interpretable SAE feature — the routing dimension.

2. **The dominant Q/K dimension is interpretable.** The routing dimension d should correspond to a monosemantic SAE feature: a direction in residual stream space that activates for a single coherent concept. The head attends to tokens that are high on this feature.

3. **The V circuit is approximately rank-1 and off-diagonal.** The effective value matrix $W_V W_O$ should have most of its singular value mass in a single component. The input direction (source dimension s) should correspond to a belief-encoding feature (dimension 0 in the formal construction); the output direction (destination dimension d) should correspond to a scratch slot in the residual stream.

4. **The written dimension is distinct from the routing dimension.** The head reads from dimension d to determine whom to attend to, and writes to dimension d' to deposit the gathered belief. These should be different SAE features — routing and evidence are separate.

What to Look for in FFN Weights

The formal OR construction uses:

W_1 row i = pattern that activates unit i, W_2 column i = belief update contributed by unit i.

With BP weights, the input pattern reads from the scratch slots (dimensions 4 and 5) and the output update writes to the belief dimension (dimension 0).

The predictions for FFN weights in a trained model:

1. W_1 **rows cluster by input feature.** Rows that read from the same SAE feature should form a cluster. Each cluster corresponds to one factor in the implicit factor graph.

2. W_2 **columns are interpretable belief updates.** Each column should correspond to a human-interpretable concept update. ROME [11] already confirms this: individual columns encode factual associations.

3. **Input and output features are related by the factor graph.** If W_1 row i reads from features A and B, then W_2 column i should update a feature C where C is a known consequence of A and B in the knowledge domain. The factor $(A, B) \to C$ is the implicit conditional probability table entry.

The Closing Experiment

The experiment that would close the loop:

1. Take a model with a trained SAE (e.g. Claude 3 Sonnet with the Anthropic SAE).

2. Identify semantic-AND heads by behavior (sharp attention to a content-determined neighbor, as in the psychic classification).

3. For each semantic-AND head, project $W_Q^T W_K$ onto the SAE feature basis. Check if the dominant singular value component corresponds to a monosemantic feature.

4. Check if the $W_V W_O$ circuit routes from a belief-encoding feature to a scratch-slot feature.

5. For each active FFN unit, identify the input SAE features (W_1 row) and the output SAE feature (W_2 column). Check if the relationship is interpretable as a conditional probability table entry.

If the results match the predictions, the formal BP construction has been directly observed in a trained model. The implicit factor graph would be legible: you could read off the factor potentials from the FFN weights and the graph structure from the attention Q/K circuits.

Why This Has Not Been Done Yet

The technical obstacle is SAE coverage. Current SAEs cover a small fraction of the residual stream's representational capacity. The routing dimensions predicted by the formal construction may not yet have clean SAE features assigned to them. As SAE coverage improves — and as the SAE feature basis becomes more complete — the experiment becomes more tractable.

The conceptual obstacle is that the field has not had a specific structural prediction to test. The formal construction provides that prediction for the first time. The weight inspection experiment is now a well-posed empirical question, not a fishing expedition.

12 The Hybrid BP Map: Four Head Types in Practice

The constructive BP mapping developed earlier applies to a specific idealized setting: sigmoid transformers with exact BP weights performing sharp index-matched routing. Production transformers differ from this in two important ways. They use activations such as GeLU or SwiGLU rather than sigmoid, and their attention heads exhibit a range of behaviors beyond sharp discrete lookup.

The right question is therefore not whether real transformers literally instantiate the constructive theorem in every component. It is whether the BP framework generates useful, testable interpretations of what trained transformers actually do. This section presents empirical head classification results and assesses what they support.

The Four Head Types

Head classification across GPT-2 small, GPT-2 medium, and Qwen 2.5 0.5B, using unsupervised attention-pattern analysis over 88 prompts with no model-specific tuning, yields four stable functional categories:

Type	Role	GPT-2 small	GPT-2 medium	Qwen 2.5 0.5
boolean-AND	Sharp routing / discrete gathering	9 (6%)	9 (2%)	29 (9%
continuous-OR	Diffuse aggregation / soft evidence mixing	28 (19%)	46 (12%)	59 (18%
hub	Global context communication	103 (72%)	305 (79%)	203 (60%
mixed	Context-dependent behavior	8 (6%)	24 (6%)	45 (13%

boolean-AND heads attend to exactly one token with near-deterministic concentration. Their behavior matches the constructive routing heads of Section 5 most directly: one neighbor's information is gathered into a specific residual stream slot before the FFN update executes.

continuous-OR heads distribute attention broadly, integrating graded evidence from across the context window rather than performing discrete lookup. These heads are not well-described by the binary BP construction, but are naturally interpretable as soft evidence aggregators operating over continuous representations.

Hub heads consistently attend to or from position 0 (the BOS token) and constitute the dominant head population across all models studied. They are best understood as a special case of boolean-AND routing: the routing key always resolves to a fixed position rather than a content-determined neighbor. This makes them discrete routers like boolean-AND heads, just attending to a shared global workspace rather than a semantically matched token. The result is a star-shaped factor graph in which position 0 functions as a global variable node, accumulating evidence from all token positions and redistributing a global context signal back.

Mixed heads switch between sharp and diffuse behavior depending on input and do not exhibit a stable functional identity across prompts. They are concentrated in late layers in every model studied — especially prominent in Qwen, where layers 22 and 23 collapse almost entirely to mixed behavior.

Coverage of the Framework

Taken together, the three well-characterized types — boolean-AND, continuous-OR, and hub — account for approximately 90% of all heads across every model studied (94% in both GPT-2 models, 87% in Qwen). Mixed heads, the genuinely uncharacterized remainder, constitute roughly 6–13% depending on architecture. The BP-derived taxonomy covers the large majority of observed head behavior across model families.

Hub as Boolean-AND at a Fixed Address

Because hub heads are a special case of boolean-AND routing, the AND and hub populations together form the discrete routing infrastructure of the network. Combining them:

	GPT-2 small	GPT-2 medium	Qwen 2.5 0.5B
discrete routing (AND + hub)	112 (78%)	314 (82%)	232 (69%)
continuous aggregation (OR)	28 (19%)	46 (12%)	59 (18%)
uncharacterized (mixed)	8 (6%)	24 (6%)	45 (13%)

Discrete routing dominates in every model. The architecture is primarily a routing system, with a minority of heads performing continuous evidence aggregation.

The Scaling of boolean-AND Heads

Within the GPT-2 family, boolean-AND head count is stable: GPT-2 small (124M parameters, 144 total heads) contains 9, and GPT-2 medium (345M parameters, 384 total heads) also

contains exactly 9. Total head count grows 2.8×; sharp routing infrastructure does not. What grows instead is hub infrastructure, from 72% to 79% of all heads.

Across architectures, the count varies: Qwen 2.5 0.5B at a comparable scale has 29 boolean-AND heads. This suggests that the number of discrete routing heads reflects architectural choices rather than a universal constant. Different model families allocate different amounts of capacity to sharp content-based routing. The consistent finding across all three models is that boolean-AND heads remain a small minority — the routing primitive is compact regardless of how many instances a given architecture deploys.

This is consistent with the BP interpretation: routing is a compact operation. What the architecture scales is inference capacity — hub infrastructure and OR aggregation — not the routing primitive itself.

Hybrid Inference

The constructive theorem establishes exact BP realization for sigmoid transformers. Production transformers use SwiGLU or GeLU. Does the BP interpretation break down?

Not entirely — but the character of the inference changes. Bayesian networks are not limited to binary variables. Continuous-variable Bayesian networks, Gaussian belief propagation, and expectation propagation are all well-established frameworks in which message passing operates over graded rather than discrete representations.

The empirical head distribution suggests a hybrid picture: boolean-AND and hub heads implement discrete routing within the BP family; continuous-OR heads implement inference-like operations over graded representations that the binary construction does not cover exactly. The constructive sigmoid theorem is then best understood as the exact limiting case of a broader family of message-passing computations that production transformers approximate.

For the boolean-AND heads specifically, the formal predictions are testable: their Q/K circuits should exhibit approximately rank-1 structure consistent with `projectDim`, and their V circuits should show `crossProject`-style routing. This is the experiment that would most directly confirm the BP interpretation in trained models.

Layer Structure

Head type distribution is not uniform across layers. GPT-2 small exhibits a three-phase structure that aligns naturally with iterative inference:

- **Layers 0–2** are dominated by continuous-OR heads performing local feature extraction. Boolean-AND heads are present but sparse. This corresponds to early rounds of BP: each node gathers evidence from nearby tokens.
- **Layer 3** is a transition layer in which all four head types are represented. Boolean-AND heads are active at exactly the point where the network shifts toward global communication.
- **Layers 4–11** are hub-dominated. The primary computation shifts to reading from and writing to the global context at position 0.

This progression — local evidence gathering followed by global integration — is what you would expect from an iterative inference system. It is consistent with the BP interpretation

but not uniquely predicted by it; other accounts of transformer computation could produce a similar layer structure.

Open Questions

Three questions mark the boundary of what the hybrid BP account currently explains:

1. **The continuous update rule.** What is the correct analog of updateBelief for continuous nodes? GeLU and SwiGLU preserve non-negativity and smooth aggregation, but their relationship to Gaussian BP or expectation propagation has not been worked out.
2. **What determines the AND head count?** Boolean-AND head count is stable within the GPT-2 family but varies across architectures — 9 in GPT-2, 29 in Qwen at comparable scale. What architectural choices drive this, and what is the minimum discrete routing infrastructure required for language modeling?
3. **Prediction vs. compatibility.** The BP framework is consistent with many observed transformer behaviors. The stronger claim — that it *predicts* them — requires identifying behaviors the framework would rule out if false. Specifying those tests is the next step toward treating the hybrid BP account as a genuine scientific hypothesis rather than an interpretive lens.

The constructive theorem gives an exact reference point for the boolean-AND case. The continuous and mixed populations remain the empirical frontier.

Part IV
Grounding and Hallucination

13 Grounded vs. Ungrounded Transformers

A grounded transformer and a standard language model may share the same underlying architecture while differing substantially in how their internal representations relate to externally defined concepts and verification procedures.

Within the BP interpretation developed in this paper, both grounded and ungrounded systems perform inference over implicit factor-graph-like structures. The key distinction is whether the graph has a formally declared interpretation.

The Distinction

A *grounded* transformer has:

- An explicit factor graph: tokens or latent states correspond to declared propositions in a known knowledge base.
- A finite concept space: the relevant propositions are bounded and specified in advance.
- A verifier: outputs can be checked against externally defined correctness criteria.

- Formal guarantees in restricted settings: on tree-structured knowledge bases with constructive BP weights, exact posterior guarantees follow from the companion theorem [3].

An *ungrounded* transformer has:

- An implicit factor graph encoded in learned parameters rather than explicitly declared structures.
- No formally declared ontology tying internal states to a finite, verified concept inventory.
- No architecture-internal correctness guarantee analogous to the grounded setting.
- Reliability that is primarily empirical rather than formally verified.

Hallucination in BP Terms

Within this framework, hallucination can be interpreted as a mismatch between model beliefs and externally verified posteriors.

For grounded systems, the comparison is well-defined:

$$b(j) \neq P_{\text{true}}(j),$$

where $P_{\text{true}}(j)$ is the correct posterior induced by the knowledge base and observed evidence.

Ungrounded language models differ because they lack a formally declared ontology and verifier. Their outputs may still correspond to meaningful or true propositions in ordinary semantic practice, but the architecture does not itself provide a mechanism guaranteeing correctness relative to a fixed external knowledge base.

From this perspective, hallucination is not merely a numerical error in the inference procedure. It is also a consequence of operating without a fully specified and verifiable grounding structure.

Scaling and Grounding

Scaling improves empirical performance. Larger models generally produce more coherent outputs, achieve lower hallucination rates on benchmarks, and exhibit richer internal representations.

However, scaling alone does not provide the same kind of formal correctness guarantees available in grounded settings. Increasing model capacity improves the quality of the learned implicit factor graph, but it does not by itself create an explicit ontology, verifier, or formally grounded concept space.

The distinction is therefore not between "working" and "non-working" systems. It is between empirical reliability and formally specified correctness guarantees.

What Grounding Would Look Like

A fully grounded transformer system would require several components:

1. A declared concept space: propositions or entities are explicitly represented and bounded.

2. A declared relational structure: dependencies between concepts are specified or verified.

3. A grounding procedure: natural language inputs are mapped into the declared representational system.

4. A verification mechanism: outputs can be checked against the underlying ontology or knowledge base.

Retrieval-augmented generation (RAG) can be viewed as a partial form of grounding because retrieved documents temporarily constrain the inference space. More complete grounding would require persistent ontological and verification structure rather than transient context retrieval alone.

The Practical Implication

The practical distinction between grounded and ungrounded systems is the kind of reliability guarantee they support.

For grounded systems in restricted settings, the constructive BP results yield exact posterior guarantees. For general-purpose language models, the strongest guarantees are typically empirical: benchmark performance, human evaluation, calibration studies, or retrieval consistency.

The gap between these two forms of reliability — formal verification versus empirical robustness — is the grounding gap.

Part V
Convergent Evidence

14 Evidence from Mechanistic Interpretability

The mechanistic interpretability literature provides some of the strongest empirical motivation for viewing transformers through a message-passing lens. Importantly, most of this work was not originally framed in terms of belief propagation or Bayesian networks. The value of the BP perspective is therefore not that it replaces the interpretability literature, but that it offers a common language for organizing many of its findings.

This section surveys several influential interpretability results and discusses how they naturally fit into a routing-and-update view of transformer computation.

Induction Heads

Olsson et al. [12] identify "induction heads": two-head circuits that implement in-context pattern matching. Head 1 (the "prefix" head) attends from position t back to the previous occurrence of the current token. Head 2 (the "induction" head) then attends to the token that followed that previous occurrence.

From the tutorial BP perspective, this resembles a two-stage gather operation. One head retrieves a historical context; the second uses that retrieved context to identify the next

relevant source of evidence. The important point is not that induction heads are "literally" BP nodes, but that the routing structure closely resembles iterative message passing over an implicit dependency graph.

Name Mover Heads and the IOI Circuit

Wang et al. [20] reverse-engineer the indirect object identification circuit in GPT-2 small. The key components are "name mover heads" that copy token identity information from one position to another. The Q/K circuits of these heads exhibit approximately rank-1 structure, while the V circuits route token-specific content into target positions.

This behavior is naturally compatible with the routing interpretation developed earlier in the paper. In the constructive BP mapping, sparse projectDim/crossProject-style operations perform a similar function: matching one feature dimension and routing information into another location in the residual stream.

The BP framework therefore provides one possible interpretation of why such sparse routing patterns repeatedly emerge in trained models.

FFN Key-Value Memories

Geva et al. [6] show that FFN layers behave like key-value memories: the first-layer weight matrix detects patterns and the second-layer matrix writes structured updates back into the residual stream. Many hidden units correspond to human-interpretable concepts or contexts.

Within the tutorial framework developed here, FFNs can be interpreted as performing local update operations conditioned on gathered evidence. The "key" determines when a particular update should activate; the "value" determines what modification is written into the evolving residual-stream state.

This interpretation does not require treating FFN rows as literal conditional-probability tables. Rather, it suggests that the key-value memory view and the message-passing view may be complementary ways of describing the same computational structure.

ROME and Localized Editing

Meng et al. [11] demonstrate that factual associations in GPT models can sometimes be edited by modifying localized FFN parameters. The resulting edits are often surprisingly surgical: changing one association without broadly disrupting neighboring behavior.

From the BP perspective, this resembles modifying a localized update rule or factor potential within a larger inference system. Again, the point is not that ROME directly proves the existence of explicit Bayes net tables inside the transformer. The point is that localized, structured update behavior is naturally compatible with a factorized message-passing interpretation.

Sparse Autoencoders and Features

Templeton et al. [18] train sparse autoencoders on Claude 3 Sonnet and recover millions of interpretable features from the residual stream. Many features exhibit coherent semantic structure, feature neighborhoods, and suppression relationships.

Within the tutorial framing, these features can be viewed as candidate nodes or directions in an implicit computational graph. Their activation values behave like evolving internal state variables whose interactions are mediated through attention and FFN updates.

The "Golden Gate Claude" experiment is especially suggestive from this perspective: clamping one feature propagates structured downstream effects through the model, much like injecting evidence into a connected message-passing system.

Attribution Graphs

Recent work from Anthropic [10] constructs attribution graphs tracing information flow between features across layers. These graphs make visible the pathways through which activations influence one another during a computation.

From the tutorial BP perspective, attribution graphs resemble partial visualizations of the model's implicit message-passing structure: attention routes information, FFNs apply transformations, and residual streams accumulate evolving state over depth.

Synthesis

The BP perspective should be understood as an organizing framework for these findings rather than a claim that interpretability researchers were unknowingly "discovering Bayes nets."

The important observation is simply that many empirical findings in mechanistic interpretability fit naturally into a routing-plus-update view of transformer computation:

- induction heads resemble multi-stage retrieval,
- routing heads move structured information between positions,
- FFNs apply localized updates,
- sparse features behave like persistent latent variables,
- and attribution graphs reveal structured pathways of influence.

The convergence between these findings and the message-passing perspective is one reason the BP framework is useful as a conceptual tool for understanding transformers.

15 Related Work

Five independent research programs have converged on the conclusion that the transformer forward pass implements belief propagation. None of them set out to prove this. Each approached the transformer from a different direction and found the same structure. This section surveys each program and its relationship to the account developed in this paper.

Belief State Geometry

Several recent papers establish that transformer residual streams represent belief states linearly. Shai et al. [17] show that the residual stream encodes a linear representation of the agent's belief state over possible world states, updated by attention operations. Piotrowski et

al. [15] show that attention implements constrained belief updates consistent with Bayesian conditioning. Agarwal et al. [1] show that transformers are primitively complete for Bayesian inference — any Bayesian computation can be implemented by a transformer.

These results are complementary to the account here. They establish the *what* (the residual stream is a belief state; attention implements Bayesian updates) without fully specifying the *how* (the specific weight construction that makes this exact, the AND/OR decomposition, the factor graph structure). The formal construction of [3] provides the missing mechanistic account.

In-Context Learning as Bayesian Inference

Xie et al. [21] argue that in-context learning is implicit Bayesian inference at the population level: the model maintains a posterior over latent document concepts and updates it as it sees more examples. This is a statistical account — it describes what ICL accomplishes in terms of the training distribution — rather than a mechanistic account of how individual forward passes implement it.

The BP account is the mechanistic complement: each forward pass is one round of BP on the implicit factor graph, and in-context learning works because the implicit factor graph encodes the conditional relationships learned from training. The two accounts are consistent and describe different levels of abstraction.

Constructive Weight Proofs

Akyürek et al. [2] and von Oswald et al. [19] show that specific weight patterns make transformers perform implicit gradient descent, explaining in-context learning as approximate optimization. The methodology is the same as [3]: construct explicit weights that implement a specific algorithm, prove correctness, confirm empirically.

The key differences: BP is exact on trees (not approximate or asymptotic), and the BP account is mechanistic per-instance (each forward pass computes a specific posterior) rather than statistical at the population level (the model behaves as if it were doing Bayesian inference in expectation over the training distribution).

GNNs, Message Passing, and Attention

Scarselli et al. [16] established that graph neural networks implement belief propagation on explicit graph structure. Gilmer et al. [7] unified GNNs under the message passing neural network (MPNN) framework. Yoon et al. [22] extended this to approximate inference. Joshi [9] observed that full self-attention is equivalent to a GNN on a complete graph.

The contribution of [3] is orthogonal: a standard transformer with full self-attention and no attention mask implements BP on an explicit *external* factor graph, where graph structure is encoded in token embeddings and routing is discovered via Q/K index matching. The graph is not hardwired into the attention mask — it is represented in the token content.

Mechanistic Interpretability

Elhage et al. [5] develop the mathematical framework for transformer circuits. Olsson et al. [12] identify induction heads as a specific named algorithm discovered by gradient descent.

Wang et al. [20] reverse-engineer the full IOI circuit in GPT-2 small. Geva et al. [6] establish the key-value memory interpretation of FFN layers. Meng et al. [11] show that factual associations can be edited by modifying FFN weights. Templeton et al. [18] find millions of monosemantic features via SAEs.

The BP account gives this literature a unified theoretical framework. Every major finding maps onto a component of the belief propagation account: induction heads are two-round BP gathers; name mover heads are projectDim/crossProject routing; FFN key-value memories are factor potentials; ROME edits are factor potential modifications; SAE features are factor graph nodes. The field has been mapping the implicit factor graph without a map. This paper provides the map.

The Log-Odds Tradition

Good [8] and Turing formalized log-odds addition as the correct algebra for combining independent binary evidence. Pearl [13] applied this algebra to graphical models via the sum-product algorithm. The connection between this tradition and the transformer architecture has not previously been made explicit before [3]. Section 2 of this paper traces the lineage and establishes that the sigmoid transformer is the mechanical implementation of the Turing-Good-Pearl algebra.

Summary

The BP account of the transformer is not a new claim in isolation. It is the convergence point of five independent research programs. The formal contribution of [3] is to make the convergence rigorous: a formally verified constructive proof that a sigmoid transformer with any weights implements weighted BP on its implicit factor graph, with explicit weights for the exact case, a uniqueness theorem showing exact posteriors force those weights, and empirical confirmation that gradient descent finds the predicted structure. This paper makes that formal account concrete and readable.

References

[1] Naman Agarwal, Siddhartha R. Dalal, and Vishal Misra. The Bayesian geometry of transformer attention. *arXiv preprint arXiv:2512.22471*, 2025. Paper I of the Bayesian Attention Trilogy.

[2] Ekin Akyürek, Dale Schuurmans, Jacob Andreas, Tengyu Ma, and Denny Zhou. What learning algorithm is in-context learning? Investigations with linear models. In *International Conference on Learning Representations*, 2023. arXiv:2211.15661.

[3] Greg Coppola. Transformers are Bayesian networks. *arXiv preprint arXiv:2603.17063*, 2026.

[4] Nelson Elhage, Tristan Hume, Catherine Olsson, et al. Toy models of superposition. *Transformer Circuits Thread*, 2022.

[5] Nelson Elhage, Neel Nanda, Catherine Olsson, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.

[6] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of EMNLP*, 2021.

[7] Justin Gilmer, Samuel S. Schütt, Patrick Riley, Oriol Vinyals, George E. Dahl, and Zoubin Ghahramani. Neural message passing for quantum chemistry. In *International Conference on Machine Learning*, 2017. arXiv:1704.01212.

[8] I.J. Good. *Probability and the Weighing of Evidence*. Charles Griffin, 1950.

[9] Chaitanya K. Joshi. Transformers are graph neural networks. *arXiv preprint arXiv:2506.22084*, 2025.

[10] Jack Lindsey et al. On the biology of a large language model. *Transformer Circuits Thread*, 2025.

[11] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, 2022.

[12] Catherine Olsson, Nelson Elhage, Neel Nanda, et al. In-context learning and induction heads. *Transformer Circuits Thread*, 2022.

[13] Judea Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, 1988.

[14] Jorge Pérez, Javier Marinković, and Pablo Barceló. On the Turing completeness of modern neural networks. *arXiv preprint arXiv:1901.03429*, 2019.

[15] Mateusz Piotrowski, Paul M. Riechers, Daniel Filan, and Adam S. Shai. Constrained belief updates explain geometric structures in transformer representations. *arXiv preprint arXiv:2502.01954*, 2025.

[16] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE Transactions on Neural Networks*, 20(1):61–80, 2009.

[17] Adam S. Shai, Sarah E. Marzen, Lucas Teixeira, Alexander Gietelink Oldenziel, and Paul M. Riechers. Transformers represent belief state geometry in their residual stream. *arXiv preprint arXiv:2405.15943*, 2024.

[18] Adly Templeton, Tom Conerly, Jonathan Marcus, et al. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024.

[19] Johannes von Oswald, Eyvind Niklasson, Ettore Randazzo, João Sacramento, Alexander Mordvintsev, Andrey Zhmoginov, and Max Vladymyrov. Transformers learn in-context by gradient descent. In *International Conference on Machine Learning*, 2023. arXiv:2212.07677.

[20] Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: A circuit for indirect object identification in GPT-2 small. In *International Conference on Learning Representations*, 2023.

[21] Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. An explanation of in-context learning as implicit Bayesian inference. In *International Conference on Learning Representations*, 2022. arXiv:2111.02080.

[22] KiJung Yoon, Renjie Liao, Yuwen Xiong, Lisa Zhang, Ethan Fetaya, Raquel Urtasun, Richard Zemel, and Xaq Pitkow. Inference in probabilistic graphical models by graph neural networks. In *ICLR Workshop on Deep Learning on Graphs*, 2019. arXiv:1803.07710.